

Docker na Prática

Disciplina: Sistemas Operacionais **Tipo:** Trabalho Prático

Formato: Individual ou em grupo **Professor:** Matheus Barquette

Descrição

Desenvolva uma aplicação e coloque ela para rodar com Docker. A complexidade da aplicação é secundária o que será avaliado é sua capacidade de **explicar e justificar cada decisão técnica**: por que essa imagem base, por que essa porta, o que acontece se remover o volume, como os serviços se comunicam.

O que entregar

São quatro entregas obrigatórias. A ausência de qualquer uma delas inviabiliza a avaliação do trabalho.

1. Aplicação

Uma aplicação funcional na linguagem de sua escolha: API REST, site HTML servido por um servidor, app Node.js, Python, Java, Go qualquer coisa que tenha uma funcionalidade visível e testável.

Requisitos:

- A aplicação deve fazer algo observável (responder a uma rota, exibir dados, processar uma entrada)
- O código deve conter comentários que explicam **decisões** de implementação, não descrições do óbvio
- Complexidade mínima é aceita o foco é o Docker, não o sistema

2. Dockerfile

O `Dockerfile` da aplicação, com cada instrução justificada via comentários no próprio arquivo.

Requisitos:

- Imagem base escolhida com justificativa (versão, tamanho, compatibilidade)
- Cada instrução comentada com o motivo da escolha, não apenas o que faz
- Porta exposta documentada e condizente com a aplicação
- `CMD` ou `ENTRYPOINT` funcional e explicado

Referência de estrutura esperada:

```
# Node 18 Alpine: imagem oficial LTS com footprint reduzido (~50MB vs ~900MB full)
FROM node:18-alpine

# Define o diretório padrão para os comandos seguintes dentro do container
WORKDIR /app

# Copia apenas o manifesto primeiro – permite que o Docker reutilize o cache
# das dependências quando só o código-fonte muda
COPY package*.json ./
RUN npm install

COPY . .

# Documenta a porta usada pela aplicação (não abre a porta por si só)
EXPOSE 3000

# CMD é substituível em runtime; ENTRYPOINT seria fixo – escolha consciente
CMD ["node", "server.js"]
```

O comentário `# instala dependências` não tem valor. O comentário que explica a estratégia de cache do `COPY package*.json` antes do `COPY . .` tem. Essa diferença será cobrada na apresentação.

3. Docker Compose

Arquivo `docker-compose.yml` com **no mínimo dois serviços** configurados e funcionando de forma integrada.

Escolha uma das combinações:

Combinação	Descrição
App + Banco de dados	PostgreSQL, MySQL ou MongoDB como segundo serviço
App + Nginx	Nginx atuando como proxy reverso na frente da aplicação

Combinação	Descrição
App + Redis	Cache em memória, demonstrando comunicação via rede interna Docker

Requisitos:

- Variáveis de ambiente declaradas (não deixar credenciais hardcoded no código da aplicação)
- Volume nomeado para persistência de dados do segundo serviço
- `depends_on` configurado corretamente
- Os dois serviços devem estar funcionando e se comunicando

Exemplo de estrutura mínima aceita:

```
version: '3.8'

services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DB_HOST=db
      - DB_NAME=meuapp
    depends_on:
      - db
    volumes:
      - ./src:/app/src

  db:
    image: postgres:15
    environment:
      - POSTGRES_PASSWORD=senha
      - POSTGRES_DB=meuapp
    volumes:
      - pgdata:/var/lib/postgresql/data # volume nomeado: dados sobrevivem ao
container

volumes:
  pgdata:
```

4. Evidências e Diário de Desenvolvimento

Documentação do processo de desenvolvimento, incluindo o que funcionou e o que não funcionou.

O que incluir:

- Print ou vídeo do `docker build` concluindo com sucesso
- Print do `docker compose up` com os serviços iniciando
- Output do `docker ps` mostrando os containers em execução
- Aplicação acessível no navegador ou via Postman/curl
- **Diário de desenvolvimento:** registro dos erros encontrados, mensagens de erro reais e como cada um foi resolvido

O diário é parte significativa da entrega. O que será avaliado não é a ausência de erros, mas a capacidade de diagnosticar e resolver.

Regras de Integridade Acadêmica

Uso de IA é permitido e esperado. Ferramentas como ChatGPT e GitHub Copilot são parte do trabalho de desenvolvimento atual. O que não é permitido é entregar algo que você não entende.

Regras:

1. **Comentários de decisão.** O código deve ter comentários que explicam escolhas, não apenas o que a linha faz.
2. **Diário com erros reais.** Sem registro de problemas enfrentados, a entrega é considerada incompleta.
3. **Código sem execução não é aceito.** Trabalho com apenas explicações textuais e sem evidências de funcionamento é inválido.
4. **Declare o uso de IA.** Inclua uma seção `## Uso de IA` no README indicando quais partes foram geradas ou assistidas por ferramentas, e o que você modificou ou aprendeu a partir disso. Não declarar quando o uso for evidente invalida a entrega.

A pergunta não é "você usou IA?". A pergunta é "você entende o que está entregando?". Isso será verificado na apresentação.

Estrutura de Entrega

O repositório ou arquivo compactado deve seguir a organização abaixo:

```
meu-projeto-docker/  
├─ src/                # código-fonte da aplicação  
├─ Dockerfile          # com comentários explicando cada instrução  
├─ docker-compose.yml  # com os dois serviços configurados  
├─ evidencias/  
│   ├─ build.png  
│   ├─ compose-up.png  
│   └─ app-rodando.png  
├─ diario.md           # erros encontrados e como foram resolvidos  
└─ README.md           # descrição do projeto + seção "Uso de IA"
```

Critérios de Avaliação

- Dockerfile funcional e comentado adequadamente
- Docker Compose funcional com dois serviços integrados
- Aplicação rodando corretamente via Docker
- Diário de desenvolvimento
- Organização e clareza da entrega